

DenseNet

DenseNet은 ResNet과 Pre-Activation ResNet보다 적은 파라미터 수로 더 높은 성능을 가진 모델.

[DenseNet의 구조]

1. Dense connectivity

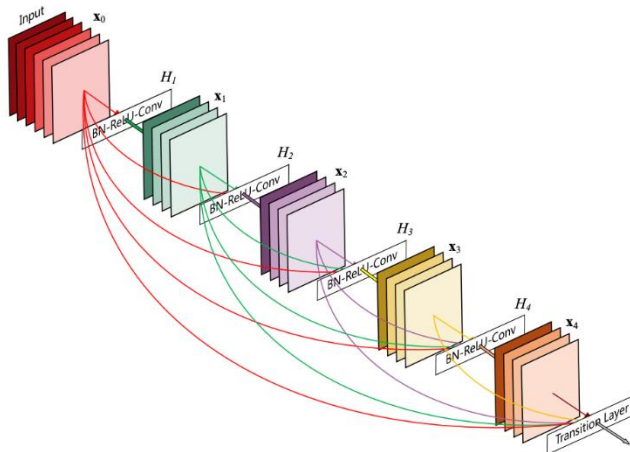
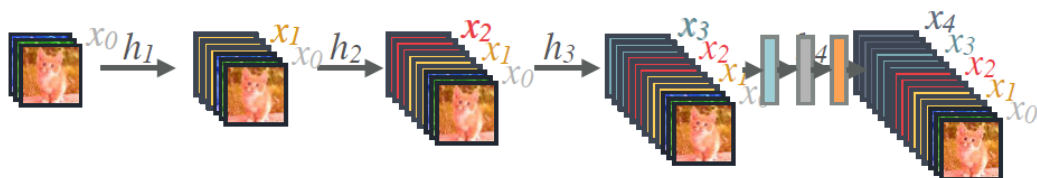


Figure 1: A 5-layer dense block with a growth rate of $k = 4$. Each layer takes all preceding feature-maps as input.

DenseNet은 이전 레이어를 모든 다음 레이어에 직접적으로 연결합니다. 따라서 **정보 흐름(information flow)**가 향상됩니다.

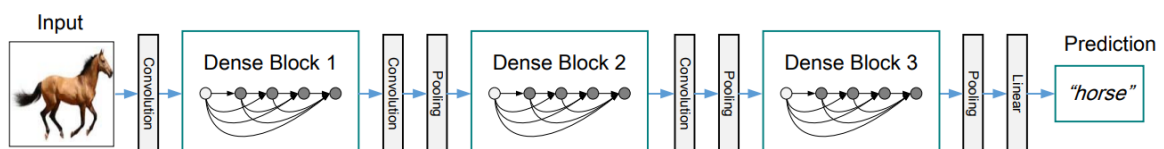
이전 layer들의 feature map을 계속해서 다음 layer의 입력과 연결하는 아이디어는 ResNet과 같은데 ResNet에서는 feature map끼리 더하기를 해주는 방식이었다면 **DenseNet에서는 feature map끼리 concatenation**을 시켜주는 것이 가장 큰 차이점이다.

아래 그림은 이전 layer의 output을 concatenation하는 것을 보여준다.



2. Dense Block

연결(concatenation) 연산을 수행하기 위해서는 피쳐맵의 크기가 동일해야 합니다. 하지만 피쳐맵 크기를 감소시키는 pooling 연산은 conv net의 필수적인 요소입니다. **pooling 연산을 위해 Dense Block 개념을 도입합니다.**



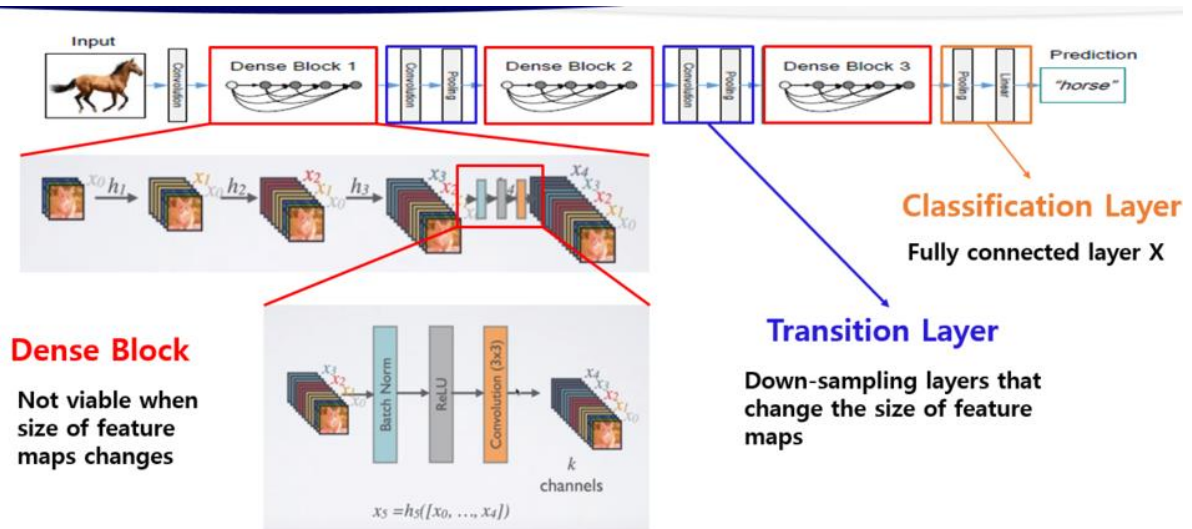
Dense Block이라고 부르는 하나의 connection 덩어리를 만들어서 전통적인 CNN처럼 Convolution ,

Pooling layer과 함께 순차적으로 Dense Block들을 거쳐서 마지막에 Linear layer후 결과를 뽑아내게 된다.

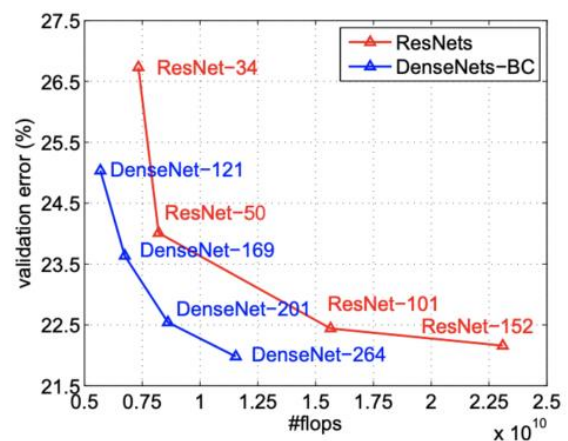
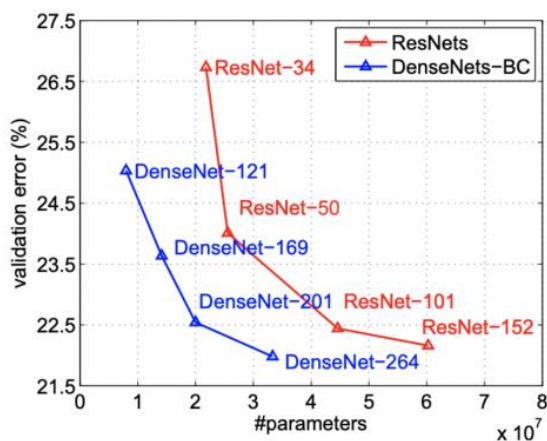
3. Advantages

- vanishing-gradient 개선
- feature propagation 강화
- Feature Reuse
- parameter 의 수 절약

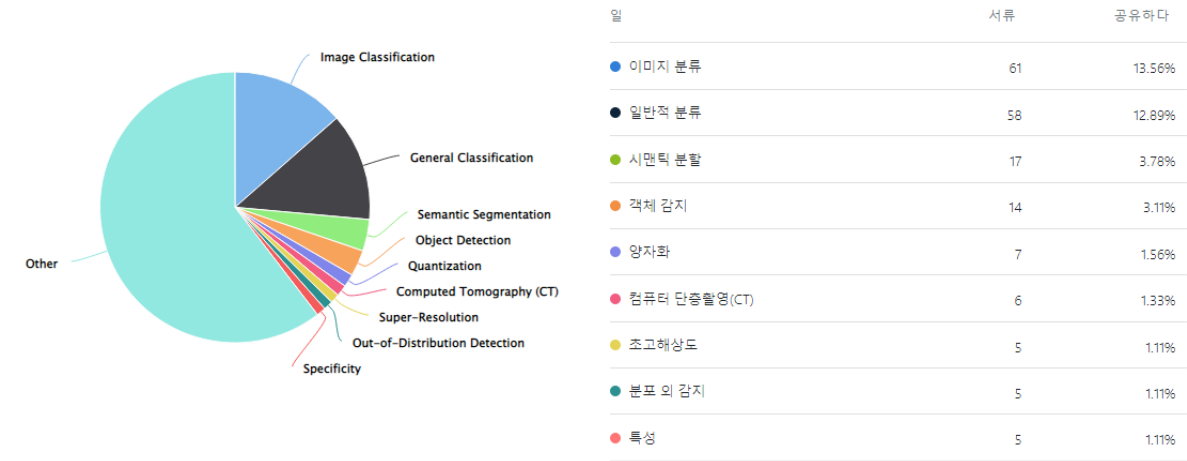
4. DenseNet 전체의 구조



5. ResNet과 DenseNet의 성능 비교

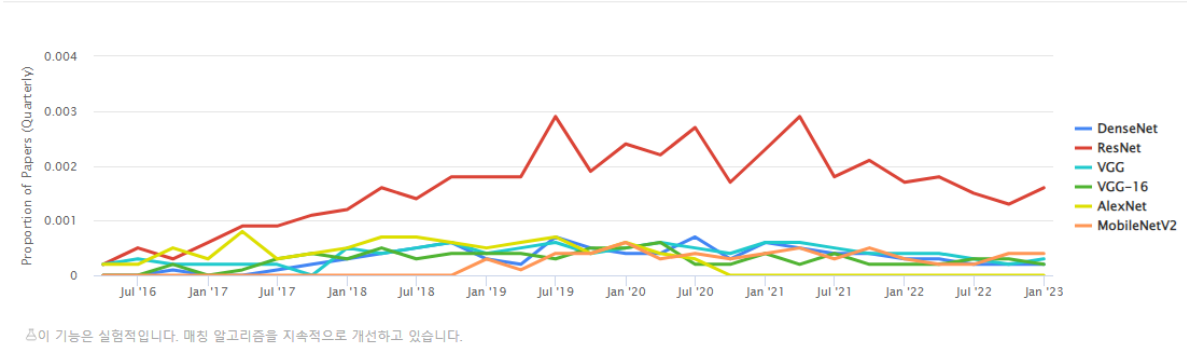


[Task]



[Usage Over Time]

시간 경과에 따른 사용량



Pytorch / densenet 참고 코드

<https://github.com/pytorch/vision/blob/6db1569c89094cf23f3bc41f79275c45e9fcb3f3/torchvision/models/densenet.py#L126>

<https://deep-learning-study.tistory.com/545> ***

<https://github.com/weiaicunzai/pytorch-cifar100/blob/master/models/densenet.py>

참고 문헌

<https://velog.io/@lighthouse97/DenseNet%EC%9D%98-%EC%9D%B4%ED%95%B4>

<https://paperswithcode.com/method/densenet>

<https://deep-learning-study.tistory.com/528>

<https://csm-kr.tistory.com/10>